

■ CRA TEAM 3 · CTRL-C

CodeFab

새로운 확장자 `.ctrlc`로 작성한 코드를 분석하고 실행하는
Interpreter

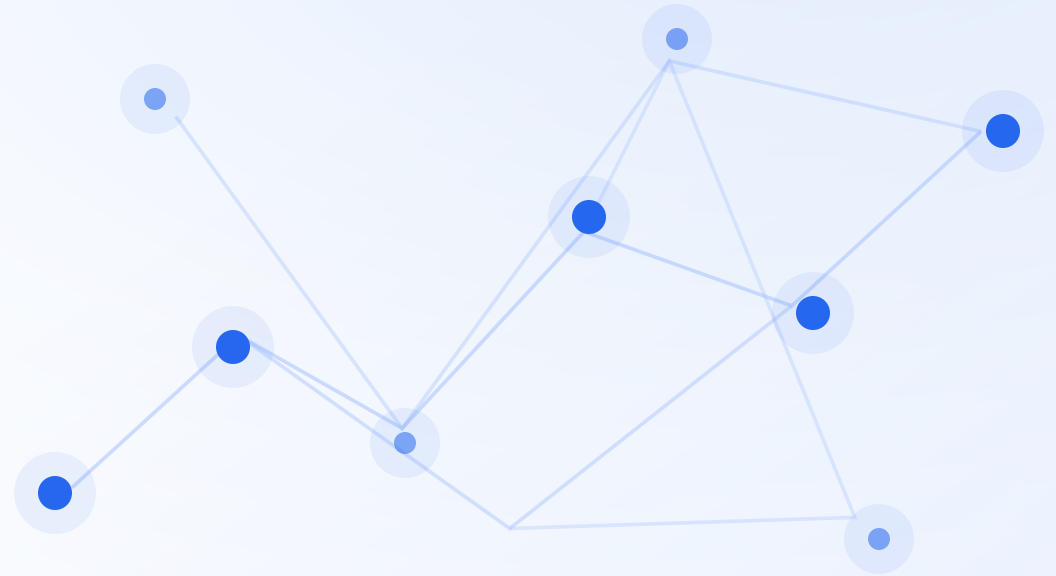
Language

Pipeline

Shell

Design Patterns

Collaboration



목차

언어 구현 **흐름**

구조, 기능, 패턴, 협업 기준 순서로
정리했습니다.

01

팀 소개

역할과 구현 분담

02

프로젝트 목표

새로운 확장자 `.ctrlc`

03

전체 구조

Shell과 interpreter pipeline

04

주요 기능

문법 · 함수 · class · import

05

디자인 패턴

AST · Callable · Command

06

협업과 품질

Review · TDD · Actions

■ TEAM

팀 역할과 구현 분담

김명준

팀장

architecture · infra · CI · docs

권소영

assembler · executor · function ·
class

김민준

expression · statement · shell

박진우

tokenizer · array · custom syntax

이예진

checker · optimizer · custom function

`.ctrlc` 실행 목표

입력

- `.ctrlc` source
- prompt 한 줄 입력
- debug 실행 대상



결과

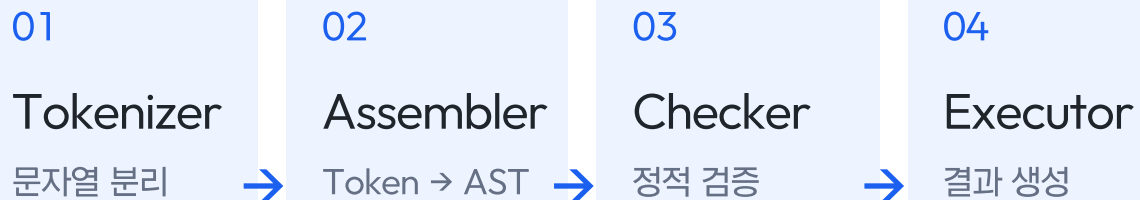
- Token / Expr / Stmt 구조화
- Checker 사전 검증
- Executor 실행 결과 생성

Shell과 Pipeline 구조

Shell Modes

- 01 Prompt Mode
한 줄 입력을 즉시 실행
- 02 File Mode
.ctrlc 파일 전체 실행
- 03 Debug Mode
Stmt 단위로 실행 상태 확인

Interpreter Pipeline



■ FEATURE IMPLEMENTATION

기능 구현 범위

각 기능은 입력, 내부 구조, 실행 결과 기준으로 설명합니다.

01

Basic

기본 문법

Stmt / Expr

Core

함수 / Class

Callable · This · Super

Module

Array / Import

데이터 · 파일 재사용

Safety

Checker

실행 전 검증

Shell

Shell Mode

Prompt · File · Debug

Stmt / Expr 실행 구조

구현 구조

Stmt는 실행 순서, Expr은 계산 결과를 담당합니다.

01 Tokenizer
source → token

02 Assembler
token → Stmt / Expr

03 Executor
Stmt 실행 · Expr 평가

[app/assembler/statement.py](#) · [app/assembler/expr.py](#)

```
var total = 0;

for (var i = 0; i < 3; i = i + 1) {
    total = total + i;
}

if (total > 2) {
    print total;
}
```

Function 실행 구조

```
Func add(a, b) {  
    return a + b;  
}  
  
print add(2, 3);
```

호출 흐름

선언은 closure와 저장하고, 호출 시 새 Environment를 만듭니다.

01 FunctionStmt
name · params · body

02 UserFunction
declaration + closure

03 call()
params binding → execute

[app/assembler/runtime.py](#)

Class / This Binding

객체 실행 흐름

class 생성과 method binding을 Callable 흐름으로 처리합니다.

01 UserClass
method table

02 UserInstance
field / method lookup

03 This binding
instance 연결

04 Super lookup
parent method 탐색

[app/assembler/runtime.py](#) · [app/executor/executor.py](#)

```
Class Robot {  
    move() { print "move"; }  
}  
  
Class SpeedRobot : Robot {  
    move() {  
        Super.move();  
        print "speed";  
    }  
}
```

Array와 Import

Static Array

고정 크기 list와 index 접근을 지원합니다.

```
Array(3)
```

```
arr[0] = value
```

```
IndexSetExpr / IndexGetExpr
```

Library Import

외부 `.ctrlc` 선언을 alias로 묶습니다.

```
import "math.ctrlc" alias math
```

Tokenizer → Assembler 재사용

```
math.add() 형태로 접근
```

Checker 실행 전 검증

ConstantFolder

계산 가능한 expression을 LiteralExpr로 치환

ScopeStack

선언, 초기화, scope distance 추적

Resolver

This, Super, return, import 규칙 확인

Scope

중복 선언 · 미초기화 변수

Class

This/Super · 자기 상속

Import

중복 import · 순환 import

Report

source line 기준 오류

Static Binding

왜 필요한가

변수 위치를 실행 전에 계산해 lookup 흐름을 단순화합니다.

선언 위치 계산

Expr에 distance 저장

Environment 직접 접근

[app/checker/scope_stack.py](#) ·
[app/executor/environment.py](#)

```
VariableExpr(name)
```

```
ScopeStack.distance_to(name)
```

```
expr.distance = depth
```

```
Environment.get_at(distance, name)
```

Shell Mode와 Command

실행 모드

01 Prompt Mode
한 줄 입력을 즉시 실행

02 File Mode
.ctrlc 파일 전체 실행

03 Debug Mode
breakpoint, step, watch로 상태 확인

Prompt 명령 처리

source line

Factory.create(source)

Command.execute(shell)

Runner → Pipeline

[app/shell/commands.py](#) · [app/shell/shell.py](#)

Custom Language Features

확장 방향

팀 전용 문법도 같은 pipeline에서 처리되도록 확장했습니다.

한글 alias

keyword token으로 정규화

`ctrl_c()`

팀 소개 native function

♡ 연산

문자열 기반 점수 계산

`.ctrlc`

file mode와 import 기준

02

■ DESIGN PATTERNS

디자인 패턴

복잡해지는 구현 지점을 패턴으로 어떻게 정리했는지 봅니다.

패턴 적용 위치

AST 구조

Composite-style AST

Stmt와 Expr node가 tree로 중첩됩니다.

expr.py · statement.py

호출 규칙

Callable Adapter

함수와 class 생성을 call로 통일합니다.

runtime.py · executor.py

생성 책임

Factory-like Creation

source를 token과 AST로 변환합니다.

tokenizer.py · assembler.py

Shell 명령

Command Pattern

prompt 입력을 command 객체로 바꿉니다.

commands.py · shell.py

Composite-style AST

적용 지점

중첩 문법을 Stmt / Expr node tree로 표현합니다.

문법 추가 위치 명확

Checker / Executor 공유

중첩 구조 표현

[app/assembler/expr.py](#) ·
[app/assembler/statement.py](#)

ForStmt

initializer: VarStmt

condition: BinaryExpr

increment: AssignExpr

body: BlockStmt([...])

Callable Adapter

```
evaluate_call()
```

```
callee.arity()
```

```
callee.call(executor, args)
```

```
Native · UserFunction · UserClass
```

적용 지점

Executor는 대상 종류와 무관하게 arity와 call만 호출합니다.

호출 검증 통합

class 생성도 call

native 확장 간단

[app/assembler/runtime.py](#) ·
[app/executor/executor.py](#)

Factory-like Creation

적용 지점

Tokenizer와 Assembler가 token과 AST 생성 책임을 가집니다.

문법 추가 경로 일정

import parser 재사용

source와 실행 분리

[app/assembler/tokenizer.py](#) ·
[app/assembler/assembler.py](#)

source text

Tokenizer.tokenize()

Token[]

Assembler.parse()

Stmt[] / Expr tree

Command Pattern

```
PromptShell.run_line()
```

```
CommandFactory.create()
```

```
Exit · Recommend · Rerun · Explain · RunCode
```

```
command.execute(shell)
```

적용 지점

Shell 입력을 Command 객체로 만들고 실행 책임을 분리합니다.

명령 책임 분리

history 정책 캡슐화

명령 추가 범위 축소

[app/shell/commands.py](#) · [app/shell/shell.py](#)

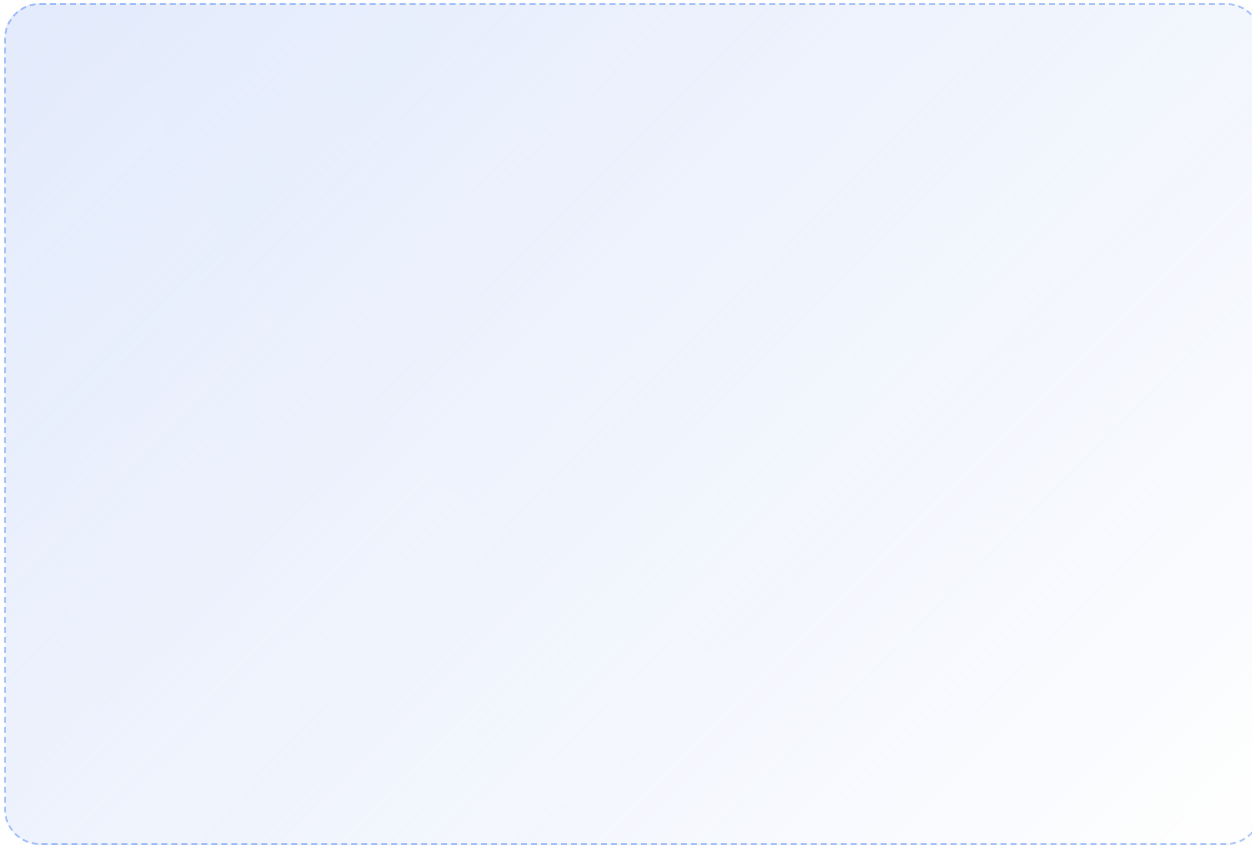
03

■ COLLABORATION

협업과 품질 기준

리뷰, 자동화, 보호 규칙으로 main 반영 기준을 맞췄습니다.

Code Review 사례



의도 공유

PR template로 변경 이유와 검증 범위를 맞췄습니다.

관점 확보

최소 2명 approve 기준으로 확인 관점을 늘렸습니다.

수정 근거

debug, import, checker 누락을 리뷰로 보강했습니다.

TDD와 회귀 테스트 사례

실패 고정

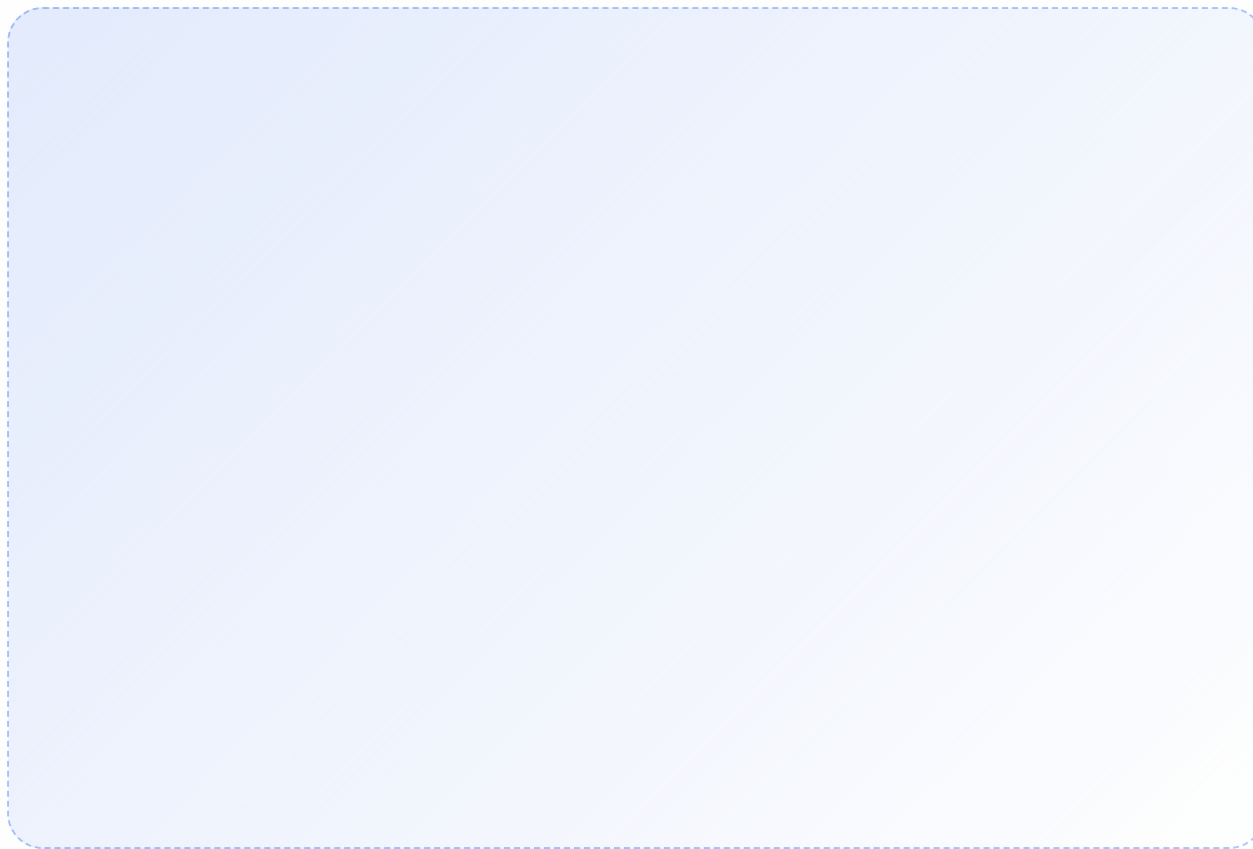
구현 전에 깨져야 하는 케이스를 먼저 남겼습니다.

회귀 방지

import, debug, checker 오류를 테스트로 관리했습니다.

자동 검증

Actions가 PR마다 같은 기준으로 실행했습니다.



Shell Command Refactoring

Before

- PromptShell 안에서 명령 분기가 커짐
- 기능 추가 시 기존 흐름을 계속 건드림
- 명령별 테스트 범위가 넓어짐

After

- 명령을 Command 객체로 분리
- PromptShell은 command 선택과 실행만 담당
- 새 shell 기능을 독립 단위로 추가

PR 작성과 검토 흐름

01

Branch

기능 단위 branch



02

PR Template

요약 · 검증 · 리뷰



03

Review

예시와 테스트 확인



04

Merge

승인 · 검사 · 병합

Actions와 Protection Rule

GitHub Actions

- black, isort, ruff로 코드 스타일 검증
- pytest와 coverage report 실행
- gist acceptance로 대표 언어 흐름 확인

Protection Rule

- main 직접 반영 방지
- approve 후 merge
- required check 통과 후 반영